Settings/general switches show high latency and global loading state #28283

Open | #28290 #28296

sahil2448 opened 8 hours ago

name: Bug report
about: Report any issues with the platform
title: "🐛 Bug: High latency and global loading state on settings switch buttons"
labels: ["🐛 bug"]
assignees: ""

Found a bug? Please fill out the sections below. 👍

Issue Summary

Namaste team! While contributing to the settings page, I noticed a small UX snag. Currently, the toggle switches (like "Search engine indexing" or "Monthly digest") are not behaving independently.

When you click any single switch, it seems like the entire group enters a "loading" state. This causes all other switches to briefly show a "not-allowed" cursor and become unresponsive until the first action is finished. The UI feels a bit heavy/laggy because one switch is blocking the others instead of handling its own state.

Steps to Reproduce

1. Go to the **Settings** page (e.g., Profile or General settings where multiple toggles exist).
2. Click on any toggle, like **"Allow search engine indexing."**
3. Immediately try to click a second toggle, like **"Monthly digest email."**
4. You will see that the second switch is "locked" or shows a blocked cursor while the first one is still processing.

Actual Results

- Clicking one switch triggers a global loading state (`isPending`) that disables all other switches in that section.
- High perceived latency—the user has to wait for the "Settings updated successfully" toast before they can interact with any other setting.

Expected Results

- Each switch should behave independently.
- Clicking one should not disable or block the others.
- Ideally, we should use **optimistic updates** so the toggle flips instantly without waiting for the network response to finish.

Technical details

- **Browser:** Chrome / Brave (Latest)
- **Node.js version:** v18.x / v20.x
- **Potential Cause:** It looks like a single `useMutation` hook's `isPending` or similar state is being passed to the `disabled` prop of every switch in the list, rather than tracking loading states per individual item.

Evidence

- I have observed that as soon as one switch is clicked, the `disabled` state is applied globally to the sibling components.
- **Testing:** Verified by trying to "double-tap" multiple settings quickly; the UI blocks the second interaction until the first request resolves.

Screen.Recording.2026-03-05.120030.mp4

@mentainers please tell whether this issue is valid..? so that i can start working on it....
I really want to work on this issue !

sahil2448 added bug 8 hours ago

sahil2448 8 hours ago Author

I found out this issue.....I would like to work on this issue...is this a valid issue to work upon???

linear bot 8 hours ago – with Linear

CAL-7289

deepshekhardas linked a pull request that will close this issue 2 hours ago

fix(settings): independent toggle loading states, unsaved changes warning, default home view #28290

EruditeCoder108 20 minutes ago

Observed Symptom
Clicking any single toggle switch on the settings page causes all other toggle switches in that section to become temporarily disabled and unresponsive until the first action (API call) is finished.

Reproduction Path
Go to the Settings page (e.g., Profile or General settings where multiple toggles exist).
Click on any toggle, like "Allow search engine indexing."
Immediately try to click a second toggle, like "Monthly digest email."
Observe that the second switch is 'locked' or shows a blocked cursor while the first one is still processing.

The `GeneralView` component defines a single `mutation = trpc.viewer.me.updateProfile.useMutation` at L44 in `general-view.tsx`.
A global loading state `isUpdateBtnLoading` is managed via `useState` at L45 in `general-view.tsx`.
This `isUpdateBtnLoading` is set to `true` at L184 before calling `mutation.mutate` and reset to `false` at L87 on `onSettled`.
`react-hook-form`'s `isSubmitting` state (accessed via `formState: { isDirty, isSubmitting }` at L144) also reflects the global form submission status for the `Form` component.
The user's description exactly matches this behavior: clicking one switch triggers a network request, causing a global loading state (`isSubmitting` or `isUpdateBtnLoading`) that then disables all other switches, showing a 'not-allowed' cursor until the request completes.
UNCERTAIN:
The exact rendering of the `SettingsToggle` components and how their `disabled` prop is passed in `general-view.tsx` could not be verified from the provided incomplete `return` block. However, the observed behavior strongly suggests a global `disabled` state propagation.

Execution Timeline
COPY
T0
User clicks a toggle switch (e.g., 'Allow search engine indexing').

T0 + very small delta
The switch's `onChange` handler is triggered, which ultimately calls `formMethods.handleSubmit`.

T0 + small delta
`setIsUpdateBtnLoading(true)` is called at `general-view.tsx` L184. `formMethods.formState.isSubmitting` also becomes `true`.

T0 + small delta
The single `mutation.mutate` (from `trpc.viewer.me.updateProfile.useMutation`) is called with the updated form values.

T0 + small delta
Due to the global loading states (`isUpdateBtnLoading` and `isSubmitting`) being `true`, any `SettingsToggle` components that have their `disabled` prop bound to these states become disabled.

T1 (during API call)
User attempts to click another toggle switch (e.g., 'Monthly digest email').

T1
The second toggle is unresponsive and displays a 'not-allowed' cursor because its `disabled` prop is `true` due to the ongoing global loading state.

T2 (network latency passed)
The API request for the first toggle completes.

T2 + small delta
`mutation.onSettled` is called, which calls `setIsUpdateBtnLoading(false)` at `general-view.tsx` L87. `formMethods.formState.isSubmitting` also becomes `false`.

T2 + small delta
All toggle switches become re-enabled.

T2 + small delta
The toast 'Settings updated successfully' is displayed.

Technical Root Cause
COPY
The root cause is that all toggle switches on the General Settings page are implicitly sharing a single global loading state (either `isSubmitting` from `react-hook-form` or `isUpdateBtnLoading` from local state) derived from a single `trpc.viewer.me.updateProfile.useMutation` hook. When any setting (including a toggle) is updated, the entire form's submission process is engaged, leading to `isSubmitting` being true and `isUpdateBtnLoading` being true for the duration of the network request. If these states are used to disable all interactive elements within the form, including other switches, it creates the observed blocking behavior.

Location: cal.com/apps/web/modules/settings/my-account/general-view.tsx
Alternative Hypotheses
The `disabled` prop of all `SettingsToggle` components is incorrectly tied to a single, global loading state (`isSubmitting` from `react-hook-form` or `isUpdateBtnLoading` from local `useState`) that tracks the progress of the entire `updateProfile` mutation.
The `reset(getValues())` call in `onSuccess` might be causing a brief re-render or state inconsistency that temporarily disables inputs.
The `SettingsToggle` component itself might be designed to disable other elements when one of its instances is active, or it's incorrectly inheriting a `disabled` prop from a parent.

```
// cal.com/apps/web/modules/settings/my-account/general-view.tsx

// ... (imports and existing code)
```

```
const GeneralView = ({ user, travelSchedules }: GeneralViewProps) => {
  // ... (existing useLocale, useSession, formMethods, etc.)

  // REMOVE:
  // const [isUpdateBtnLoading, setIsUpdateBtnLoading] = useState<boolean>(false);

  // Rename the existing mutation to be clearer about its role for the main form fields
  const mainFormMutation = trpc.viewer.me.updateProfile.useMutation({
    onSuccess: async (res) => {
      await utils.viewer.me.invalidate();
      revalidateSettingsGeneral();
      revalidateTravelSchedules();
      reset(getValues()); // Keep this for resetting the form after a full save
      showToast(t("settings_updated_successfully"), "success");
      await update(res);

      if (res.locale) {
        window.calNewLocale = res.locale;
        document.cookie = `calNewLocale=${res.locale}; path=/`;
      }
    },
    onError: () => {
      showToast(t("error_updating_settings"), "error");
    },
    onSettled: async () => {
      await utils.viewer.me.invalidate();
      revalidateSettingsGeneral();
      revalidateTravelSchedules();
      // REMOVE: setIsUpdateBtnLoading(false);
    },
  });

  // NEW: Create individual mutations for each toggle
  const updateDynamicBookingMutation = trpc.viewer.me.updateProfile.useMutation({
    onSuccess: () => {
      utils.viewer.me.invalidate();
      revalidateSettingsGeneral();
      showToast(t("settings_updated_successfully"), "success");
    },
    onError: () => {
      showToast(t("error_updating_settings"), "error");
      // Optionally revert the UI state here if optimistic update was used
    },
  });

  const updateSEOIndexingMutation = trpc.viewer.me.updateProfile.useMutation({
    onSuccess: () => {
      utils.viewer.me.invalidate();
      revalidateSettingsGeneral();
      showToast(t("settings_updated_successfully"), "success");
    },
    onError: () => {
      showToast(t("error_updating_settings"), "error");
    },
  });

  const updateMonthlyDigestMutation = trpc.viewer.me.updateProfile.useMutation({
    onSuccess: () => {
      utils.viewer.me.invalidate();
      revalidateSettingsGeneral();
      showToast(t("settings_updated_successfully"), "success");
    },
    onError: () => {
      showToast(t("error_updating_settings"), "error");
    },
```

```
});

const updateBookerEmailVerificationMutation = trpc.viewer.me.updateProfile.useMutation({
  onSuccess: () => {
    utils.viewer.me.invalidate();
    revalidateSettingsGeneral();
    showToast(t("settings_updated_successfully"), "success");
  },
  onError: () => {
    showToast(t("error_updating_settings"), "error");
  },
});

// ... (existing state for individual toggles)
const [isAllowDynamicBookingChecked, setIsAllowDynamicBookingChecked] = useState(
  !!user.allowDynamicBooking
);
const [isAllowSEOIndexingChecked, setIsAllowSEOIndexingChecked] = useState(
  user.organizationSettings?.allowSEOIndexing === false
    ? !!user.organizationSettings?.allowSEOIndexing
    : !!user.allowSEOIndexing
);
const [isReceiveMonthlyDigestEmailChecked, setIsReceiveMonthlyDigestEmailChecked] = useState(
  !!user.receiveMonthlyDigestEmail
);
const [isRequireBookerEmailVerificationChecked, setIsRequireBookerEmailVerificationChecked] = useS
  !!user.requiresBookerEmailVerification
);

// Update the Form handleSubmit for the main form (locale, timezone, etc.)
// The 'isDisabled' for the main save button should still use isDirty and mainFormMutation.isPendi
const isDisabled = mainFormMutation.isPending || !isDirty;

// ... (inside the return statement)
return (
  // ...
  <Form
    form={formMethods}
    handleSubmit={async (values) => {
      // REMOVE: setIsUpdateBtnLoading(true);
      mainFormMutation.mutate({ // Use mainFormMutation here
        ...values,
        locale: values.locale.value,
        timeFormat: values.timeFormat.value,
        weekStart: values.weekStart.value,
      });
    }}>
    {/* ... other form fields ... */}

    {/* This is an illustrative example. You'll need to locate where SettingsToggle is actually re
    {/* Each SettingsToggle should now trigger its own specific mutation and use its own loading s
    <div>
      <SettingsToggle
        label={t("allow_dynamic_booking")}
        checked={isAllowDynamicBookingChecked}
        onCheckedChange={(checked) => {
          setIsAllowDynamicBookingChecked(checked); // Optimistic update
          updateDynamicBookingMutation.mutate({ allowDynamicBooking: checked });
        }}
        disabled={updateDynamicBookingMutation.isPending} // Use its own pending state
      />
      <SettingsToggle
        label={t("allow_seo_indexing")}
        checked={isAllowSEOIndexingChecked}
        onCheckedChange={(checked) => {
```

```
            setIsAllowSEOIndexingChecked(checked); // Optimistic update
            updateSEOIndexingMutation.mutate({ allowSEOIndexing: checked });
          }}
          disabled={updateSEOIndexingMutation.isPending} // Use its own pending state
        />
        <SettingsToggle
          label={t("receive_monthly_digest_email")}
          checked={isReceiveMonthlyDigestEmailChecked}
          onCheckedChange={(checked) => {
            setIsReceiveMonthlyDigestEmailChecked(checked); // Optimistic update
            updateMonthlyDigestMutation.mutate({ receiveMonthlyDigestEmail: checked });
          }}
          disabled={updateMonthlyDigestMutation.isPending} // Use its own pending state
        />
        <SettingsToggle
          label={t("require_booker_email_verification")}
          checked={isRequireBookerEmailVerificationChecked}
          onCheckedChange={(checked) => {
            setIsRequireBookerEmailVerificationChecked(checked); // Optimistic update
            updateBookerEmailVerificationMutation.mutate({ requiresBookerEmailVerification: checked
          }}
          disabled={updateBookerEmailVerificationMutation.isPending} // Use its own pending state
        />
      </div>

      <SectionBottomActions
        primaryBtnText={t("update")}
        primaryBtnToolTip={isDisabled ? t("no_changes") : ""}
        primaryBtnProps={{
          type: "submit", // This will trigger the mainFormMutation
          disabled: isDisabled, // Use mainFormMutation's pending state and dirty state for the main
          isLoading: mainFormMutation.isPending, // Use mainFormMutation's pending state for its own
        }}
        // ...
      />
    </Form>
  );
};
```

By creating a separate `trpc.viewer.me.updateProfile.useMutation` hook for each independent toggle switch, each toggle gains its *own* `isPending` state. When a user interacts with a specific toggle, only that toggle's mutation is triggered, and its `isPending` state becomes `true`, disabling *only that individual toggle* during its API request. Other toggles, being bound to their own independent mutation states, remain interactive and responsive. This allows for concurrent updates and improves the perceived performance and user experience by eliminating the global blocking behavior. The main form's submission button can still use a separate mutation and its `isPending` state for its own loading indication.

dougwithseismic added a commit that references this issue 8 minutes ago

`fix: use per-toggle loading state in general settings (calcom#28283)` ... `c690a25`

dougwithseismic mentioned this

fix: use per-toggle loading state in general settings #28296

Add a comment

Write Preview

Use Markdown to format your comment

Paste, drop, or click to add files

Close issue Comment

Remember, contributions to this repository should follow its contributing guidelines, security policy and code of conduct.

Open Settings/general switches show high latency and global loading state #28283